

현재 프로젝트 기반 임베디드·시스템 SW 직무와 지원을 위한 학습 로드맵

실행 요약

현재 프로젝트를 **AES-XTS + FPGA + AXI/DMA + Linux + Storage**로 한정해 다시 직무 시장과 대조했다. 기존 조사에서 도출한 “FPGA 암호화 프로젝트”보다 **HW/SW 경계의 System Software·Linux Device Driver·BSP·PCIe/CXL·Storage Firmware 프로젝트로 포지셔닝해야** 한다는 결론은 그대로 유효하다. 특히 기존 보고서의 9~16쪽에서 제시한 **Linux Device Driver → DMA → PCIe → Storage/NVMe** 학습축은 현재 공고들과도 일치한다.

이번 재조사에서 가장 중요한 추가 발견은 **XCENA의 신입 SW 공고**다. XCENA는 신입 직무 하나에서 PCIe/CXL configuration, cache coherency, ATS, DMA, Embedded Linux, bootloader, CXL driver, NVMe SSD를 함께 명시하고 있다. 따라서 현재 프로젝트와의 직접 연결성만 놓고 보면 **DEEPX System SW와 XCENA PCIe/CXL·BSP를 최상위 학습 기준 JD로 삼는** 것이 합리적이다. ¹

현재 프로젝트를 기준으로 추천 직무 우선순위는 다음과 같다.

우선순위	직무군	적합도	지금 추가해야 할 핵심
최우선	Linux Device Driver / System SW	96/100	kernel module, MMIO, IRQ, DMA API, Device Tree
최우선	BSP / SoC Platform SW	94/100	boot flow, U-Boot, DT, ARM/SoC 초기화, driver
최우선~상	PCIe/CXL / Accelerator Driver·Firmware	92/100	PCIe BAR, MSI/MSI-X, DMA/IOMMU, PCI driver
상	Storage Firmware / SSD System SW	86/100	NAND, FTL, NVMe, firmware 구조
중상	Embedded Security Firmware/BSP	80/100	XTS + key management + secure boot 개념
중상	FPGA Platform / HW Acceleration	76/100	timing/SG/descriptor/host SW integration

이 점수는 객관적인 시장 통계가 아니라 **프로젝트 직접 중첩 35%, 현재 채용시장 요구 20%, 신입 진입성 20%, 다른 직무로의 재사용성 15%, 포트폴리오 증명 용이성 10%**로 구성된 분석용 점수다. 특히 FPGA 직무가 낮아서가 아니라, 현재 사용자가 이미 가진 FPGA 지식을 **순수 RTL 경력 경쟁으로 가져가는 것보다 Linux/System SW 쪽으로 확장할 때 취업 가능한 회사의 폭이 더 넓다**는 의미다. 현재 DEEPX 신입 SW 직무는 ROM boot, bootloader, BSP와 USB·PCIe·DDR·SD/eMMC driver까지 포함하고, XCENA 신입 SW 직무도 PCIe/CXL·DMA와 Embedded Linux/BSP를 동시에 다룬다. ²

결론적으로 지금 AES 자체를 더 깊게 파는 것은 우선순위가 아니다.

현재 프로젝트 → Linux kernel driver → DMA API → IRQ/MMIO → PCIe → IOMMU → Storage/NVMe

이 한 줄을 완성한 뒤, 지원 직무에 따라 BSP·Storage·CXL·Security 중 하나만 깊게 들어가는 것이 가장 효율적이다.

직무를 실제 채용공고에서 역산하면 무엇을 요구하는가

Linux Device Driver와 System SW

가장 직접적인 목표다. DEEPX의 현재 신입 SW Engineering 공고에서 System SW 분야는 ROM boot code, bootloader, BSP, USB·PCIe·DDR·SD/eMMC·MIPI 등의 driver, Power Management/MMU, SoC verification을 명시한다. 공고 자체가 경력 무관이며, 별도 현재 채용 목록에서도 Entry-level Engineering이 노출돼 있다. ³

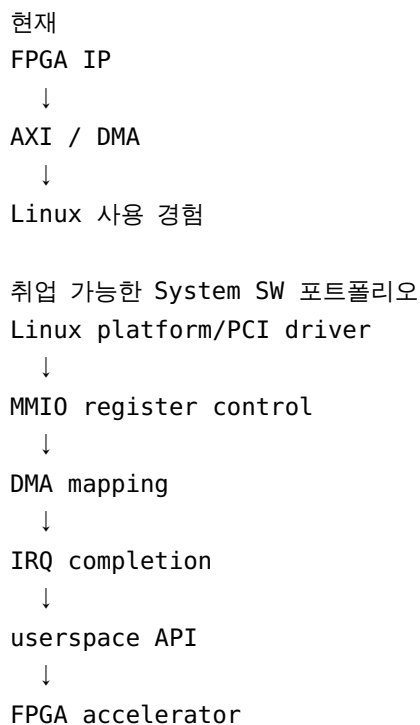
DEEPX를 기준으로 보면 사용자가 가진 `FPGA + AXI/DMA + Linux + Storage`는 상당히 좋은 출발점이다.

다만 “DMA를 사용했다”와 “Linux DMA driver를 작성했다”는 전혀 다른 증거다. **지원 전에 반드시**

`probe → MMIO → DMA buffer → IRQ → userspace` 흐름을 실제 Linux driver 코드로 하나 완성하는 것이 좋다.

FuriosaAI의 현재 `Systems Software Engineer, PCIe Device Driver`는 그 다음 단계의 좋은 기준이다. 실제 업무가 Linux PCIe device driver/kernel module 개발, DMA/IOMMU, interrupts, BAR mapping, userspace API, zero-copy data path이며, 최소자격에도 Linux kernel internals, DMA/IOMMU, C kernel module, PCIe architecture, gdb/fttrace/perf 등이 들어간다. ⁴

즉 다음 차이가 핵심이다.



BSP와 SoC Platform SW

BSP는 현재 프로젝트를 버리지 않고 진입하기 좋은 두 번째 축이다.

XCENA의 신입 SW 채용은 이 방향을 매우 명확하게 보여준다. BSP 파트에서 EEPROM/SPI Flash bootloader, ARM Cortex Embedded Linux porting, DRAM/SPI/I2C/UART 초기화, CXL interface driver, NVMe SSD memory expansion, Secure Boot/Debug를 제시한다. 동시에 같은 신입 공고에서 PCIe/CXL firmware의 configuration, cache coherency, ATS, DMA까지 다룬다. ⁵

경력용 BSP 공고를 보면 그 이후 요구 수준도 보인다. XCENA의 경력 BSP 직무는 Bootloader, ARM Embedded Linux, CXL driver, NVMe SSD, Secure Boot, Linux kernel driver 경험을 요구하고 있다. 즉 **신입에서 무엇을 배우고 몇 년 뒤 어디까지 가는지가 한 회사 안에서 매우 잘 드러난다.** ⁶

BSP 지원을 위해서는 단순히 “Linux를 쓸 줄 안다”가 아니라 아래 흐름을 설명할 수 있어야 한다.

```
Reset
↓
ROM / first stage
↓
Bootloader
↓
Device Tree
↓
Linux kernel
↓
platform_device
↓
platform_driver.probe()
↓
MMIO / IRQ / DMA
```

여기서 지금 프로젝트와 직접 연결되는 부분은 마지막 세 단계다. 따라서 부트로더부터 새 프로젝트를 만드는 것보다 **현재 FPGA accelerator를 Linux platform device로 표현하고 Device Tree → driver binding을 붙이는 작업이 먼저다.**

PCIe/CXL와 Accelerator System Software

이 직무는 현재 프로젝트와 기술적으로 가장 깊게 연결되지만, 회사에 따라 경력 장벽이 높다.

XCENA 신입 공고는 PCIe/CXL configuration, device management, cache coherency, ATS, DMA를 명시하므로 현재 기준으로 드문 신입 진입로다. 경력 PCIe/CXL firmware 직무에서는 PCIe/CXL configuration/capability/status, power state/error reporting, address translation, cache coherency, DMA, compliance까지 요구한다. ⁷

FuriosaAI는 host-side Linux driver 쪽으로 같은 문제를 바라본다. Linux kernel, PCIe BAR, DMA/IOMMU, interrupts, zero-copy가 핵심이다. ⁴ Rebellions도 현재 공식 Careers에 **Linux Device Driver Engineer**, **NPU Kernel Driver Engineer**, **SoC System Software Engineer**, **Embedded**

Software Engineer - NPU/RTOS&Embedded Linux 를 동시에 채용 직군으로 노출하고 있어 accelerator 기업에서 이 기술군이 별도의 핵심 SW 직무로 존재함을 확인할 수 있다. ⁸

따라서 처음부터 CXL 전체를 공부해서는 안 된다.

PCIe보다 CXL을 먼저 공부하는 것은 순서가 거꾸로다.

먼저 다음 네 가지만 이해한다.

PCIe device 발견 → BAR/MMIO → DMA → MSI/MSI-X

그 후:

IOMMU → ATS → cache coherency → CXL

순서가 좋다.

Storage Firmware와 SSD System SW

Storage는 AES-XTS가 가장 자연스럽게 의미를 갖는 분야다.

Silicon Motion의 현재 한국 Application Engineer 공고는 NAND/controller/SSD/eMMC/UFS/SD, SATA/PCIe SSD testing, chipset failure analysis를 요구하며 관련 경력 3년 이상이다. Firmware와 C/C++ 경험도 우대한다. 즉 즉시 신입 타깃이라기보다는 **현재 프로젝트를 Storage 방향으로 발전시킬 때 어떤 도메인 지식이 필요한지 보여주는 JD다.** ⁹

Sandisk의 최근 역할들도 같은 기술 연결을 보여준다. 서울의 NAND System Architecture 인턴은 NAND, PCIe/UCle/CXL, NoC, DMA, Firmware를 함께 다루며, 글로벌 Linux PCIe Driver Developer 직무에서는 PCIe driver, DMA engine, MMIO, mailbox를 명시한다. 서울 인턴은 PhD candidate 조건이므로 일반 신입 지원용 공고로 볼 수 없지만, **Storage 회사에서도 NAND와 PCIe/DMA/system architecture가 한 시스템으로 연결된다는 기술적 증거**로는 유용하다. ¹⁰

따라서 Storage 준비 순서 역시:

NAND 특성

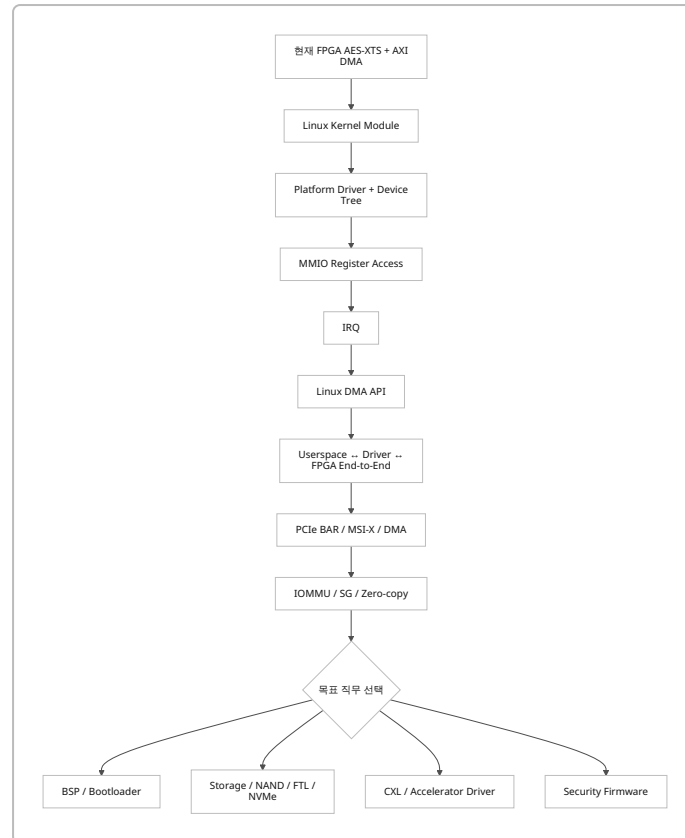
- FTL의 필요성
- LBA와 physical NAND mapping
- NVMe host interface
- PCIe
- DMA
- XTS / storage security

로 이해해야 한다.

AES부터 시작해서 NAND로 가는 것이 아니다.

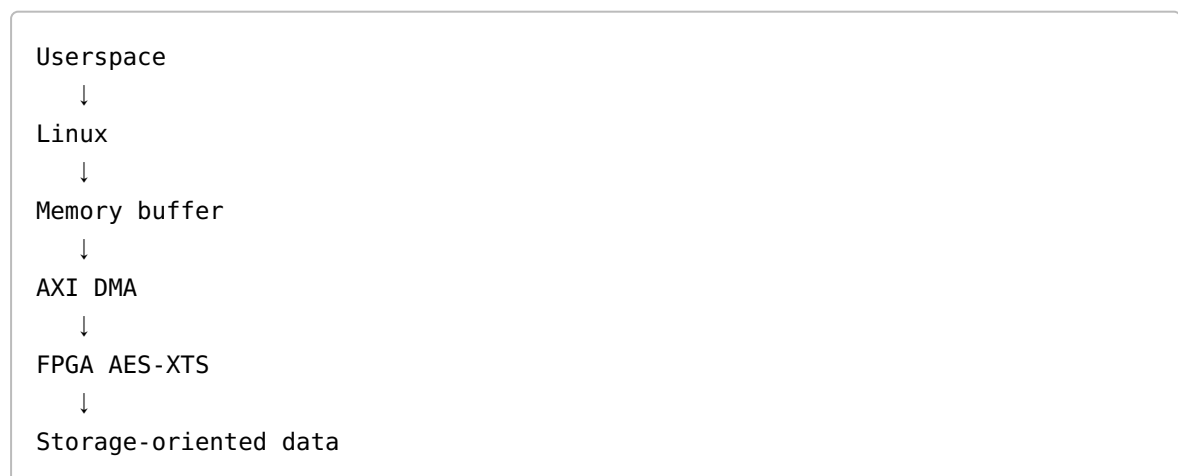
공통 핵심 학습 로드맵

아래는 10~12주 정도의 집중 학습을 가정한 순서다. 기간은 절대적인 기준이 아니다. 각 단계의 “통과 조건”을 만족하면 다음 단계로 이동하는 방식이 더 중요하다.



기반 정리

먼저 현재 프로젝트를 한 장짜리 시스템 그림으로 그린다.



이 시점에 새 기능을 추가하지 않는다.

반드시 답할 수 있어야 하는 것은:

- CPU가 무엇을 하고 FPGA가 무엇을 하는가.
- DMA를 왜 사용했는가.
- 데이터가 어느 메모리에서 어느 메모리로 이동하는가.
- 어떤 레지스터를 SW가 제어하는가.
- 완료를 polling 하는가 interrupt로 받는가.
- XTS의 data unit이 프로젝트에서 무엇에 대응하는가.

업로드된 기존 조사에서도 “새 프로젝트를 하나 더 만드는 것보다 현재 프로젝트에 boot → kernel → driver → DMA → FPGA 경로를 붙이는 편이 낫다”는 결론을 내렸다. [filecite](#)turn0file0

Linux kernel module과 driver model

첫 구현 단계다.

배워야 할 것은 `module_init/module_exit`, `kbuild`, `struct device`, `platform_driver`, `probe/remove`, resource 관리 정도다. Linux 공식 문서는 platform device가 SoC 내부 주변장치처럼 CPU에서 직접 주소 지정 가능한 장치를 표현하며, driver가 등록되고 device와 match되면 `probe()`가 호출되는 구조를 설명한다.

11

실습 통과 조건

현재 FPGA IP를 대상으로:

```
insmod
→ driver probe
→ register resource 확보
→ MMIO register read/write
→ rmmmod
```

까지 동작한다.

처음에는 DMA도 IRQ도 넣지 않는다.

읽을 문서:

1. [Linux — Building External Modules](#)

외부 kernel module을 빌드하는 기본 문서다. `make -C ... M=$PWD` 방식부터 익힌다. 12

2. [Linux — Device Drivers](#)

Linux driver model의 기본 구조. 13

3. [Linux — Platform Devices and Drivers](#)

FPGA/SoC peripheral을 Linux driver와 연결할 때 가장 먼저 읽어야 한다. 14

Device Tree와 MMIO

다음은 하드웨어 정보를 driver source에 hard-code하지 않는 단계다.

Linux Device Tree 문서는 DT가 OS가 읽을 수 있는 형태로 hardware를 기술하며 bus, interrupt, GPIO, peripheral 등의 정보를 표현한다고 설명한다. DT node가 platform device로 생성되어 platform driver와 bind되는 과정도 설명돼 있다. ¹⁵

통과 조건

```
Device Tree
reg = <...>
interrupts = <...>
↓
platform_driver.probe()
↓
resource 획득
↓
ioremap
↓
FPGA control/status register access
```

가 설명되고 실제 동작한다.

읽을 문서:

[Linux and the Devicetree](#) ¹⁵

여기서 Device Tree Specification 전체를 암기할 필요는 없다.

IRQ

다음으로 FPGA의 `done` 신호를 Linux interrupt로 받아본다.

Linux IRQ API는 `request_irq()` 또는 관련 인터페이스를 통해 interrupt handler를 등록하는 구조다. ¹⁶

통과 조건

CPU가 polling loop를 돌지 않아도:

```
FPGA operation complete
→ IRQ
→ kernel handler
→ completion/wakeup
→ userspace 확인
```

이 가능해야 한다.

읽을 문서:

처음에는 MSI-X, affinity, threaded IRQ까지 확장하지 않는다.

Linux DMA API

여기가 가장 중요한 단계다.

Linux DMA 문서가 설명하는 핵심은 **CPU virtual address와 device가 실제로 사용하는 DMA address가 같다고 가정해서는 안 된다는 것이다**. `dma_map_*`() 이 필요한 이유 중 하나가 IOMMU가 DMA address를 physical memory에 매핑할 수 있기 때문이다. 17

이 개념을 이해하지 않고 `virt_to_phys()` 나 임의 물리 주소를 넘기면 “Linux device driver를 안다”고 보기 어렵다.

먼저 구분해야 한다.

```
CPU virtual address
    ≠
CPU physical address
    ≠
DMA address / IOVA
```

그리고:

```
coherent DMA
streaming DMA
scatter-gather
cache coherency
```

순으로 배운다.

통과 조건

userspace 요청이:

```
userspace
→ driver
→ DMA-safe buffer
→ FPGA
→ interrupt
→ driver
→ userspace
```

로 실제 왕복한다.

읽을 문서:

[Linux — Dynamic DMA Mapping Guide](#) 17

현재 AMD/Xilinx AXI DMA를 사용한다면 반드시 제조사 문서도 병행한다. AXI DMA는 Memory-Mapped와 AXI4-Stream 간 data movement를 제공하며 optional Scatter/Gather를 지원한다. AMD 문서에는 simple DMA에서 control/address/length register를 설정하고 완료 시 interrupt를 받는 programming sequence도 명시돼 있다.

18

[AMD AXI DMA Product Guide PG021](#) 19

특히 다음 순서로 읽는다.

Overview → Typical System Interconnect → Register Map → Direct Register Mode → Descriptor Management → Scatter/Gather

AMD의 descriptor 문서는 SG mode에서 SW가 descriptor chain과 data buffer를 관리하고 HW가 descriptor를 fetch/process/update하는 관계를 설명한다. 이 모델은 이후 PCIe/NVMe queue나 accelerator command queue를 이해할 때도 매우 유용하다. 20

Memory ordering과 동기화

DMA가 “가끔 된다” 수준에서 “왜 항상 맞게 동작하는지 설명할 수 있다”로 넘어가는 단계다.

CPU/compiler/device는 메모리 접근 순서를 사용자가 생각하는 그대로 항상 관찰하게 해주지 않으며, Linux는 DMA-capable device와 공유 메모리를 다룰 때 `dma_wmb()`, `dma_rmb()` 등의 ordering primitive를 제공한다. 21

여기서는 lock-free algorithm을 공부하려는 것이 아니다.

통과 조건

다음 질문에 답할 수 있으면 충분하다.

“descriptor 내용을 memory에 쓰고 device doorbell을 울릴 때 왜 ordering을 생각해야 하는가?”

읽을 문서:

[Linux Kernel Memory Barriers](#) 21

PCIe

이제 PCIe로 간다.

Linux 공식 PCI driver guide가 제시하는 device 초기화 흐름 자체를 먼저 외운다.

```
device enable
→ MMIO resource request
→ DMA mask
```

- DMA buffer
- IRQ
- hardware engine enable

Linux PCI 문서도 PCI device를 driver가 획득한 후 MMIO resource, DMA mask, IRQ, DMA engine 등의 초기화가 이어진다고 설명한다. ²²

반드시 이해할 것:

- Root Complex
- Endpoint
- Configuration Space
- BAR
- MMIO
- Bus Mastering
- DMA
- MSI / MSI-X

아직 공부하지 말 것

PHY, LTSSM 세부 state, equalization, lane margining, SerDes 회로설계.

읽을 문서:

[Linux — How To Write Linux PCI Drivers](#) ²²

이 단계가 끝나면 현재 프로젝트 설명을:

“AXI 기반 FPGA accelerator를 Linux에서 제어해 본 경험을 바탕으로 PCIe endpoint의 BAR/MMIO와 host DMA/interrupt 모델까지 확장해 학습했다.”

라고 연결할 수 있다.

IOMMU와 고성능 data path

PCIe와 DMA가 먼저 되고 나서 시작한다.

FuriosaAI가 현재 DMA/IOMMU와 zero-copy를 요구하는 것이 대표적이다. ⁴

먼저 개념만:

Device DMA address
↓
IOMMU

↓
System physical memory

그리고:

scatter-gather → pinned memory → IOVA → zero-copy

정도로 확장한다.

처음부터 VFIO/IOMMUFD implementation을 파지 않는다. 나중에 필요하면 Linux IOMMUFD 문서를 보면 된다. IOMMUFD는 userspace에서 I/O address space와 I/O page table을 관리하기 위한 API를 제공한다. ²³

[Linux IOMMUFD](#) ²³

디버깅과 성능 측정

이 단계까지 프로젝트가 동작하면 새 기능보다 측정이 중요하다.

최소 측정:

payload size
throughput
DMA setup latency
accelerator latency
IRQ completion latency
CPU usage

그리고 failure case:

bad length
timeout
DMA error
unexpected IRQ
FPGA reset

를 재현한다.

FuriosaAI가 kernel debugging 도구로 gdb, ftrace, perf를 직접 명시하는 것도 같은 이유다. ⁴ Linux ftrace는 kernel function/event tracing을 위한 tracefs 기반 기능을 제공한다. ²⁴

읽을 문서:

[Linux ftrace](#) ²⁴

이 시점 이후에야 프로젝트는 단순 데모가 아니라 **system software portfolio**가 된다.

목표 직무별 분기 학습과 참고 문서

공통과정이 끝난 뒤 **한 번에 한 갈래만** 선택하는 것을 권한다.

DEEPX형 System SW를 노릴 때

현재 DEEPX 신입 SW Engineering의 System SW 부분은 bootloader/BSP, PCIe·DDR·SD/eMMC driver, PM/MMU와 SoC verification까지 포함한다. 25

학습 순서는:

```
Linux driver
→ DMA/IRQ
→ PCIe
→ Device Tree
→ Boot flow
→ SD/eMMC 개념
→ ARM/MMU 기초
```

다.

DEEPX 지원용으로는 **NAND FTL**을 깊게 파는 것보다 **PCIe + kernel driver**가 먼저다.

핵심 문서:

- [Linux Platform Driver](#) 14
- [Linux DMA API](#) 17
- [Linux PCI Driver Guide](#) 22
- [Linux Device Tree](#) 15
- [DEEPX 신입 SW Engineering](#) 25

XCENA형 PCIe/CXL·BSP를 노릴 때

현재 프로젝트와 가장 흥미로운 연결이다.

학습 순서는:

```
Linux DMA
→ PCIe
→ PCIe configuration/BAR
→ IOMMU/Address Translation
→ NVMe
→ cache coherency 개념
→ CXL
```

이다.

CXL을 PCIe보다 먼저 공부하지 않는다.

XCENA 신입 공고가 ATS, cache coherency, DMA까지 언급하지만 이는 학습의 최종 목적지이지 시작점이 아니다.

5

문서:

- [XCENA SW 개발 신입](#) 5
- [Linux PCI Driver Guide](#) 22
- [NVM Express Base Specification](#) — 현재 NVMe 2.4 specification set은 2026년 8월 공개됐다. 26
- [CXL — About CXL](#) — 처음에는 full spec보다 이 개요부터 보는 것이 낫다. CXL은 processor, memory expansion, accelerator 사이의 cache-coherent interconnect로 설명된다. 27
- [CXL Specification Landing Page](#) — 현행 specification 접근 페이지. 28

CXL 4.0 전체 specification은 처음부터 읽을 필요가 없다. 실제 implementation 단계가 되어야 필요하다. 현재는 PCIe와 coherency/address translation의 관계를 설명하는 수준이면 충분하다.

FuriosaAI · Rebellions형 Accelerator Driver를 노릴 때

여기서는 공통과정을 더 깊게 간다.

```
PCI driver
→ DMA API
→ Scatter/Gather
→ IOMMU
→ IRQ/MSI-X
→ pinned memory
→ zero-copy
→ concurrency
→ profiling
```

FuriosaAI의 현행 공고가 거의 이 순서 그대로이며, Rebellions 역시 현재 Linux Device Driver/NPU Kernel Driver/System SW 직무를 운영하고 있다. 29

이 단계부터 읽을 자료:

- [Linux DMA API](#) 17
- [Linux Memory Barriers](#) 21
- [Linux IOMMUFD](#) 23
- [Linux ftrace](#) 24
- [FuriosaAI PCIe Device Driver JD](#) 4
- [Rebellions Careers](#) 8

DPDK, RDMA, SPDK는 이후다. FuriosaAI에서 우대하지만 core qualification보다 먼저 학습할 이유가 없다. 4

Storage Firmware를 노릴 때

공통과정 뒤에 다음 순서다.

NAND

- page / block / erase-before-write
- FTL
- mapping / GC / wear leveling
- NVMe
- firmware architecture
- power failure / consistency
- XTS / security

FTL이 필요한 근본 이유는 NAND의 erase-before-write 특성 때문이다. 고전적인 FTL survey는 mapping 방식과 FTL의 핵심 문제를 체계적으로 설명한다. ³⁰

자료:

- [A Survey of Flash Translation Layer](#) ³⁰
- [NVM Express Base Specification](#) ²⁶
- [SPDK Concepts](#) — NVMe를 어느 정도 이해한 뒤 보는 고성능 storage system 자료다. SPDK 문서에는 userspace driver, DMA, NVMe I/O, NAND SSD internals가 별도 개념 문서로 정리돼 있다. ³¹
- [SPDK NVMe Driver](#) — submission/completion queue를 코드/API 관점에서 확인하기 좋다. ³²
- [FMMU paper](#) — FTL address translation 일부를 HW로 넘기는 연구로, 사용자의 HW accelerator 프로젝트와 Storage를 연결해 보는 데 특히 좋다. 저자들은 mapping 병목을 hardware automation으로 옮겨 평가했다. ³³

처음부터 새로운 FTL 알고리즘을 만들려고 하지 않는다.

면접에서는:

“NAND는 in-place update가 어려워 logical-to-physical mapping과 GC/wear leveling이 필요하다.”

를 설명할 수 있으면 첫 단계는 충분하다.

AES-XTS와 Security Firmware를 노릴 때

XTS는 현재 프로젝트의 **차별화 요소**지 학습의 중심축은 아니다.

현재 최종 NIST SP 800-38E는 XTS-AES를 storage device의 confidentiality를 위한 방식으로 승인하며 authentication을 제공하지 않는다고 명시한다. 2026년 9월 3일 NIST가 Revision 1 초안을 공개했으며, 현재 날짜 기준 이 문서는 아직 **Initial Public Draft**이고 기존 2010판이 final publication이다. Rev.1은 IEEE Std 1619-2025를 참조하고 data-unit/key-scope 요구사항 등을 명확화하는 방향이다. ³⁴

따라서 공부 순서는:

AES 기본

- XTS가 필요한 이유
- tweak
- data unit / sector / LBA 관계
- 두 key의 역할

- ciphertext stealing 개념
- XTS는 integrity를 제공하지 않음
- key management / secure boot는 별도 문제

정도면 충분하다.

문서:

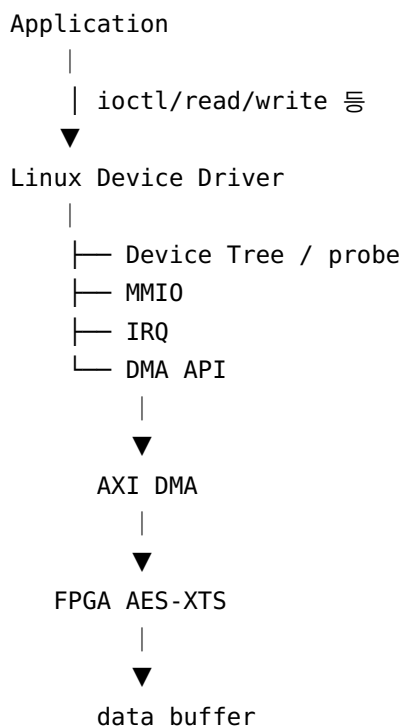
- [NIST SP 800-38E Final](#) ³⁵
- [NIST SP 800-38E Revision 1 관련 공지](#) ³⁶
- [NIST Block Cipher Techniques](#) ³⁷
- [NIST CAVP Block Cipher Modes](#) — XTS-AES validation test 자료를 찾을 출발점으로 적합하다. ³⁸

지금 하지 않을 것: cryptanalysis, formal proof, 새로운 XTS 변형 설계.

포트폴리오를 어떻게 완성해야 하는가

학습을 “읽었다”로 끝내면 채용에서는 큰 가치가 없다. 각 단계는 현재 프로젝트에 한 번씩 연결돼야 한다.

최종 포트폴리오의 목표 구조는 다음 정도가 적절하다.



여기에 README에는 네 가지가 들어가면 된다.

첫째, Architecture. CPU, memory, DMA, FPGA가 어떤 역할을 하는지 한 페이지에 표시한다.

둘째, Data path.

```

userspace buffer
→ kernel
→ DMA
→ FPGA
→ DMA
→ completion IRQ
→ userspace

```

를 주소 공간과 함께 설명한다.

셋째, Measurement. DMA 사용 전후, payload size별 throughput/latency, CPU utilization 중 가능한 것만 측정한다. 측정하지 않은 수치를 만들지 않는다.

넷째, Known limitations.

예를 들면:

```

현재 AXI 기반이며 실제 PCIe endpoint 구현은 하지 않음
IOMMU는 개념/host-side 학습까지 수행
NAND FTL은 구현하지 않음
XTS는 confidentiality만 제공
production key management는 구현 범위 밖

```

라고 명확하게 적는 것이 오히려 좋다.

면접에서는 다음 질문에 답할 수 있어야 한다.

질문	목표 답변 수준
왜 DMA인가?	CPU memcopy를 data movement에서 분리하고 accelerator와 memory 간 transfer를 효율화하기 위해
DMA address와 physical address가 같은가?	항상 그렇지 않으며 IOMMU가 개입할 수 있으므로 Linux DMA API를 사용
BAR는 무엇인가?	PCIe device resource를 host address space에 노출하는 창
MSI/MSI-X는 왜 쓰는가?	PCIe device가 host에 interrupt를 전달하는 메커니즘
Device Tree는 왜 필요한가?	board/SoC hardware description을 driver code에서 분리
FTL은 왜 필요한가?	NAND의 erase-before-write와 address mapping 문제를 숨기기 위해
NVMe에서 중요한 구조는?	controller/namespaces와 submission/completion queue
XTS는 왜 storage에 쓰나?	고정 크기 data unit의 confidentiality에 적합
XTS가 integrity도 보장하나?	아니다
현재 프로젝트와 production 차이는?	PCIe/IOMMU, error recovery, power failure, key management, FTL, validation 등이 추가됨

Linux DMA API가 DMA address/IOMMU mapping을 추상화한다는 점, NIST가 XTS를 storage confidentiality 용도로 정의하면서 authentication을 제공하지 않는다고 명시한 점은 이 면접 답변의 핵심 기술적 근거다. 39

실제 지원 우선순위와 학습 운영 원칙

현재 공개된 직무와 프로젝트 중첩만 놓고 보면 **지원·공부 대상을 동일하게 취급하면 안 된다.**

즉시 지원 또는 공고 원문 재확인 대상: DEEPX의 현재 SW Entry-level Engineering은 System SW 안에서 bootloader/BSP/PCIe·SD/eMMC driver를 명시하며 경력 무관이다. XCENA의 SW 개발 신입 역시 PCIe/CXL/DMA와 Embedded Linux/BSP/NVMe를 한 번에 포함한다. 두 공고가 현재 프로젝트를 가장 적게 버리고 지원 언어로 변환할 수 있는 직무다. 2

중기 목표 JD: FuriosaAI PCIe Device Driver는 DMA/IOMMU/BAR/zero-copy/kernel debugging까지 올라간 모습을 보여주므로 현재 공부의 다음 목표로 적합하다. Rebellions의 현행 채용 목록도 Linux Device Driver, NPU Kernel Driver, SoC System SW, Embedded Linux/RTOS 계열이 별도 직무군으로 존재한다. 29

Storage 중기 목표: Silicon Motion의 한국 AE는 3년 이상 경력을 요구하므로 즉시 신입 타깃보다 NAND/SSD/NVMe 공부 범위를 역산하는 기준으로 쓰는 편이 낫다. 9

가장 중요한 것은 **모든 분야를 병렬로 공부하지 않는 것이다.**

추천 시간 배분은:

공통 Linux driver / DMA / IRQ	50%
PCIe	20%
목표 직무 분기	20%
AES-XTS 및 기존 프로젝트 정리	10%

정도다.

특히 Storage를 목표로 하지 않는데 FTL 논문을 깊게 읽거나, CXL 회사에 지원하지 않는데 CXL 4.0 전체 specification을 읽거나, accelerator driver 기초가 없는데 DPDK/RDMA/SPDK부터 들어가는 것은 학습 투자 대비 효율이 낮다.

가장 먼저 완성할 한 가지 결과물은 이것이다.

현재 AES-XTS FPGA accelerator를 Linux platform/device driver에서 MMIO로 제어하고, Linux DMA API를 이용해 buffer를 전달하며, 완료를 IRQ로 받아 userspace까지 돌려주는 end-to-end demo.

이 한 가지가 완성되면 DEEPX의 System SW, XCENA의 BSP/PCIe-CXL, FuriosaAI·Rebellions의 accelerator driver, 이후 Storage/NVMe 계열까지 동일한 프로젝트를 기반으로 확장할 수 있다. 현재 채용공고들이 실제로 요구하는 기술 구조와도 가장 직접적으로 겹친다. 40

불확실성과 조사 시점

조사 기준일은 **2026년 9월 9일**이다. FuriosaAI와 Rebellions 공식 페이지는 현재 검색 시점에 현행 채용 목록으로 확인됐다. DEEPX와 XCENA의 개별 페이지는 공식 ATS에 존재하고 지원 정보를 포함하지만 검색 색인의 crawl 시점은

약 3개월 전으로 표시된 자료도 있으므로 실제 지원 직전에는 해당 공식 Careers 페이지에서 접수 상태를 다시 확인해야 한다. 41

또한 기존 PDF에서 언급된 개별 공고 가운데 일부는 이미 마감되거나 역할이 바뀌었을 가능성이 있다. 따라서 이번 보고서에서는 기존 보고서의 기술 매핑은 유지하되, 특정 회사의 “현재 지원 가능” 판단에는 최신 공식 페이지를 우선했다. 기존 보고서는 당시 프로젝트 범위를 AES·XTS / FPGA / AXI·DMA / Linux / Storage로 제한하고, 실제로 해보지 않은 PCIe kernel driver, IOMMU, NAND FTL 경험을 임의로 보유 역량으로 확장하지 않는 원칙을 사용했다. 이 원칙은 지원서 작성에서도 그대로 유지하는 것이 안전하다. fileciteturn0file0

가장 최신 표준 측면에서는 NIST SP 800-38E Revision 1은 2026년 9월 3일 공개된 초안이며, 2026년 9월 9일 현재 final로 취급해서는 안 된다. NVMe는 NVM Express가 2026년 8월 4일 NVMe 2.4 specification set을 공개했으며, 학습 단계에서는 최신 spec 전체를 외우기보다 host↔controller 구조와 queue model을 우선 이해하는 편이 목적에 맞다. 42

1 5 7 SW 개발 신입 채용

https://xcena.career.greetinghr.com/en/o/111727?utm_source=chatgpt.com

2 3 25 40 [SW] Entry-level Engineering (Bachelor's/Master's/PhD/전문연구요원)

https://deepx.career.greetinghr.com/ko/o/208096?utm_source=chatgpt.com

4 29 41 Furiosa.ai

https://furiosa.ai/career/systems-software-engineer-pcie-device-driver-4005793201?utm_source=chatgpt.com

6 Embedded system BSP 개발

https://xcena.career.greetinghr.com/en/o/110624?utm_source=chatgpt.com

8 Careers - Rebellions

<https://kr.rebellions.ai/careers/>

9 Application Engineer (AE)-Silicon Motion

<https://www.siliconmotion.com/company/careers/3PV/detail>

10 Sandisk PhD Intern - NAND System Architecture | SmartRecruiters

https://jobs.smartrecruiters.com/Sandisk/744000145627449-phd-intern-nand-system-architecture?utm_source=chatgpt.com

11 14 Platform Devices and Drivers — The Linux Kernel documentation

https://docs.kernel.org/driver-api/driver-model/platform.html?utm_source=chatgpt.com

12 Building External Modules — The Linux Kernel documentation

https://docs.kernel.org/6.2/kbuild/modules.html?utm_source=chatgpt.com

13 Device Drivers — The Linux Kernel documentation

https://docs.kernel.org/driver-api/driver-model/driver.html?utm_source=chatgpt.com

15 Linux and the Devicetree — The Linux Kernel documentation

https://docs.kernel.org/devicetree/usage-model.html?utm_source=chatgpt.com

16 Linux generic IRQ handling — The Linux Kernel documentation

https://docs.kernel.org/5.19/core-api/genericirq.html?utm_source=chatgpt.com

17 39 Dynamic DMA mapping Guide — The Linux Kernel documentation

https://docs.kernel.org/6.5/core-api/dma-api-howto.html?utm_source=chatgpt.com

18 Direct Register Mode (Simple DMA) - Direct Register Mode (Simple DMA) - 7.1 English - PG021

https://docs.amd.com/r/en-US/pg021_axi_dma/Direct-Register-Mode-Simple-DMA?utm_source=chatgpt.com

- 19 Documentation - Documentation - 7.1 English - PG021
https://docs.amd.com/r/en-US/pg021_axi_dma/Documentation?utm_source=chatgpt.com
- 20 Typical System Interconnect - Typical System Interconnect - 7.1 English - PG021
https://docs.amd.com/r/en-US/pg021_axi_dma/Typical-System-Interconnect?utm_source=chatgpt.com
- 21 Linux kernel memory barriers — The Linux Kernel documentation
https://docs.kernel.org/core-api/wrappers/memory-barriers.html?utm_source=chatgpt.com
- 22 1. How To Write Linux PCI Drivers — The Linux Kernel documentation
https://docs.kernel.org/PCI/pci.html?utm_source=chatgpt.com
- 23 IOMMUFD — The Linux Kernel documentation
https://docs.kernel.org/6.12/userspace-api/iommufd.html?utm_source=chatgpt.com
- 24 ftrace - Function Tracer — The Linux Kernel documentation
https://docs.kernel.org/6.12/trace/ftrace.html?utm_source=chatgpt.com
- 26 NVMe Express® Base Specification - NVMe Express
https://nvmexpress.org/specification/nvm-express-base-specification/?utm_source=chatgpt.com
- 27 About CXL® - Compute Express Link
https://computeexpresslink.org/about-cxl/?utm_source=chatgpt.com
- 28 CXL Specification Landing Page - Compute Express Link
https://computeexpresslink.org/cxl-specification-landing-page/?utm_source=chatgpt.com
- 30 A survey of Flash Translation Layer - ScienceDirect
https://www.sciencedirect.com/science/article/abs/pii/S1383762109000356?utm_source=chatgpt.com
- 31 SPDK: Storage Performance Development Kit
https://spdk.io/doc/?utm_source=chatgpt.com
- 32 SPDK: NVMe Driver
https://spdk.io/doc/nvme.html?utm_source=chatgpt.com
- 33 FMMU: A Hardware-Automated Flash Map Management Unit for Scalable Performance of NAND Flash-Based SSDs
https://arxiv.org/abs/1704.03168?utm_source=chatgpt.com
- 34 35 SP 800-38E, Recommendation for Block Cipher Modes of Operation: the XTS-AES Mode for Confidentiality on Storage Devices | CSRC
https://csrc.nist.gov/pubs/sp/800/38/e/final?utm_source=chatgpt.com
- 36 42 NIST Releases SP 800-38Er1 for Comments | CSRC
https://csrc.nist.gov/news/2026/nist-releases-sp-800-38er1-for-comments?utm_source=chatgpt.com
- 37 Block Cipher Techniques | CSRC
https://csrc.nist.gov/Projects/Block-Cipher-Techniques/BCM?utm_source=chatgpt.com
- 38 Cryptographic Algorithm Validation Program | CSRC
https://csrc.nist.gov/Projects/Cryptographic-Algorithm-Validation-Program/CAVP-TESTING-BLOCK-CIPHER-MODES?utm_source=chatgpt.com